

Evidencia de Pruebas de API — ReqRes

Proyecto: Makers QA – Prueba Técnica
Módulo: Módulo 3 – API Testing
Autor: Francisca Bravo
Fecha de ejecución: 22/06/2026
Herramienta: Cypress 15 + mochawesome reporter
API bajo prueba: ReqRes (<https://reqres.in/api>)

Entorno de ejecución

Variable	Valor
Base URL	https://reqres.in/api
Autenticación	Header <code>x-api-key</code> (free tier)
Framework	Cypress 15
Archivos de prueba	api/automated-tests/POST.cy.js · GET.cy.js · PUT.cy.js · PATCH.cy.js · DELETE.cy.js
Fixtures	cypress/fixtures/api-data.json
Reporte HTML	cypress/reports/html/output.html

Nota sobre el comportamiento de ReqRes: ReqRes es una API mock de sólo lectura. Los usuarios creados con `POST /users` no persisten en el dataset real (IDs disponibles: 1–12). Este comportamiento está documentado en los casos donde aplica.

Resumen de ejecución

ID	Método	Endpoint	Escenario	Resultado
TC-API-001	POST	/users	Crear usuario → 201, body y Content-Type válidos	PASS
TC-API-002	POST→GET	/users/{id}	Flujo encadenado: crear y consultar	PASS
TC-API-003	POST	/users	<code>createdAt</code> en formato ISO 8601	PASS
TC-API-004	POST	/users	Tiempo de respuesta < 3000ms	PASS
TC-API-005	POST	/users	Sin campo <code>name</code> → 201	PASS

ID	Método	Endpoint	Escenario	Resultado
TC-API-006	POST	/users	Sin campo job → 201	PASS
TC-API-007	POST	/users	Body vacío → 201	PASS
TC-API-008	POST	/users	Tipos incorrectos → 201	PASS
TC-API-009	GET	/users?page=1	Lista paginada → 200 + estructura válida	PASS
TC-API-010	GET	/users/2	Usuario existente → 200 + datos completos	PASS
TC-API-011	GET	/users/9999	ID inexistente → 404 + body vacío	PASS
TC-API-012	GET	/users/abc	ID no numérico → 404	PASS
TC-API-013	PUT	/users/2	Actualización completa → 200 + updatedAt	PASS
TC-API-014	PATCH	/users/2	Actualización parcial → 200 + campo correcto	PASS
TC-API-015	DELETE	/users/2	Eliminar → 204 sin cuerpo	PASS

Total: 15 casos · **PASS:** 15 · **FAIL:** 0

Detalle por caso de prueba

TC-API-001 · POST /users — Crear usuario (happy path)

Archivo: `api/automated-tests/POST.cy.js`

Request

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>

{
  "name": "Test User",
```

```
"job": "Automation Engineer"
}
```

Response

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
```

```
{
  "name": "Test User",
  "job": "Automation Engineer",
  "id": "732",
  "createdAt": "2026-06-22T14:31:05.243Z"
}
```

Assertiones verificadas

- `status` igual a 201
- `body.name` igual a "Test User"
- `body.job` igual a "Automation Engineer"
- `body.id` existe y no está vacío
- `body.createdAt` existe
- `Content-Type` incluye `application/json`

Resultado: PASS

TC-API-002 · POST /users → GET /users/{id} — Flujo encadenado

Archivo: `api/automated-tests/POST.cy.js`

Request 1 — Crear usuario

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>
```

```
{
  "name": "Test User",
  "job": "Automation Engineer"
}
```

Response 1

```
HTTP/1.1 201 Created
```

```
{
  "name": "Test User",
  "job": "Automation Engineer",
  "id": "733",
  "createdAt": "2026-06-22T14:31:06.810Z"
}
```

Request 2 — Consultar por el id recibido

```
GET https://reqres.in/api/users/733
x-api-key: <api_key>
```

Response 2

```
HTTP/1.1 404 Not Found
```

```
{}
```

Assertiones verificadas

- **POST status** igual a **201**
- **body.id** existe (el id se extrae dinámicamente)
- **GET status** es **200** o **404** (ReqRes no persiste usuarios creados: IDs del mock van del 1 al 12)

Comportamiento documentado: ReqRes devuelve 404 al hacer GET del id generado por POST porque ese id no existe en el dataset real. Este es el comportamiento esperado y la aserción lo contempla explícitamente.

Resultado: PASS

TC-API-003 · POST /users — Validar formato ISO 8601 en **createdAt**

Archivo: `api/automated-tests/POST.cy.js`

Request

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>
```

```
{
  "name": "Test User",
  "job": "Automation Engineer"
}
```

Response

```
HTTP/1.1 201 Created
```

```
{
  "name": "Test User",
  "job": "Automation Engineer",
  "id": "734",
  "createdAt": "2026-06-22T14:31:07.512Z"
}
```

Assertiones verificadas

- `body.createdAt` cumple la expresión regular: `^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d+Z$`
- El valor `"2026-06-22T14:31:07.512Z"` coincide con el formato ISO 8601

Resultado: PASS

TC-API-004 · POST /users — Tiempo de respuesta inferior a 3000ms

Archivo: `api/automated-tests/POST.cy.js`

Request

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>
```

```
{
  "name": "Test User",
```

```
"job": "Automation Engineer"
}
```

Response

```
HTTP/1.1 201 Created
Duration: ~320ms
```

Assertiones verificadas

- `res.duration` menor a 3000 ms

Resultado: PASS

TC-API-005 · POST /users sin campo `name` — Comportamiento de mock

Archivo: `api/automated-tests/POST.cy.js`

Request

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>
```

```
{
  "job": "Automation Engineer"
}
```

Response

```
HTTP/1.1 201 Created
```

```
{
  "job": "Automation Engineer",
  "id": "735",
  "createdAt": "2026-06-22T14:31:08.901Z"
}
```

Assertiones verificadas

- `status` igual a 201
- `body.id` existe

Observación: ReqRes no valida la presencia de campos requeridos. En una API de producción este caso debería retornar `400 Bad Request`. Comportamiento documentado como propio del mock.

Resultado: PASS

TC-API-006 · POST /users sin campo `job` — Comportamiento de mock

Archivo: `api/automated-tests/POST.cy.js`

Request

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>
```

```
{
  "name": "Test User"
}
```

Response

```
HTTP/1.1 201 Created
```

```
{
  "name": "Test User",
  "id": "736",
  "createdAt": "2026-06-22T14:31:09.441Z"
}
```

Assertiones verificadas

- `status` igual a 201
- `body.id` existe

Observación: Mismo comportamiento que TC-API-005. Mock acepta body parcial sin error.

Resultado: PASS

TC-API-007 · POST /users con body vacío

Archivo: `api/automated-tests/POST.cy.js`**Request**

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>
```

```
{}
```

Response

```
HTTP/1.1 201 Created
```

```
{
  "id": "737",
  "createdAt": "2026-06-22T14:31:10.002Z"
}
```

Assertiones verificadas

- `status` igual a `201`
- `body.createdAt` existe

Observación: ReqRes crea un registro aunque el body esté vacío. Comportamiento propio del mock documentado.

Resultado: PASS

TC-API-008 · POST /users con tipos de datos incorrectos

Archivo: `api/automated-tests/POST.cy.js`**Request**

```
POST https://reqres.in/api/users
Content-Type: application/json
x-api-key: <api_key>
```



```
{  
  "name": 99999,  
  "job": true  
}
```

Response

HTTP/1.1 201 Created

```
{  
  "name": 99999,  
  "job": true,  
  "id": "738",  
  "createdAt": "2026-06-22T14:31:10.553Z"  
}
```

Assertiones verificadas

- `status` igual a `201`

Observación: El mock no realiza validación de tipos. En producción debería retornar `400 Bad Request` o `422 Unprocessable Entity`. Comportamiento documentado.

Resultado: PASS

TC-API-009 · GET /users?page=1 — Lista paginada

Archivo: `api/automated-tests/GET.cy.js`

Request

```
GET https://reqres.in/api/users?page=1  
x-api-key: <api_key>
```

Response

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

```
{
  "page": 1,
  "per_page": 6,
  "total": 12,
  "total_pages": 2,
  "data": [
    {
      "id": 1,
      "email": "george.bluth@reqres.in",
      "first_name": "George",
      "last_name": "Bluth",
      "avatar": "https://reqres.in/img/faces/1-image.jpg"
    },
    {
      "id": 2,
      "email": "janet.weaver@reqres.in",
      "first_name": "Janet",
      "last_name": "Weaver",
      "avatar": "https://reqres.in/img/faces/2-image.jpg"
    }
  ],
  "support": {
    "url": "https://contentcaddy.io?utm_source=reqres&utm_medium=json&utm_campaign=referral",
    "text": "Tired of writing endless social media content? Let Content Caddy generate it for you."
  }
}
```

Assertiones verificadas

- `status` igual a 200
- `body.page` igual a 1
- `body.total` existe
- `body.data` es un array no vacío
- `Content-Type` incluye `application/json`

Resultado: PASS

TC-API-010 · GET /users/2 — Usuario existente

Archivo: `api/automated-tests/GET.cy.js`

Request

```
GET https://reqres.in/api/users/2
x-api-key: <api_key>
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

```
{
  "data": {
    "id": 2,
    "email": "janet.weaver@reqres.in",
    "first_name": "Janet",
    "last_name": "Weaver",
    "avatar": "https://reqres.in/img/faces/2-image.jpg"
  },
  "support": {
    "url": "https://contentcaddy.io?
utm_source=reqres&utm_medium=json&utm_campaign=referral",
    "text": "Tired of writing endless social media content? Let Content Caddy
generate it for you."
  }
}
```

Assertiones verificadas

- `status` igual a 200
- `body.data.id` igual a 2
- `body.data.email` existe
- `body.data.first_name` existe
- `body.data.last_name` existe
- `body.data.avatar` existe

Resultado: PASS

TC-API-011 · GET /users/9999 — ID inexistente → 404

Archivo: `api/automated-tests/GET.cy.js`

Request

```
GET https://reqres.in/api/users/9999
x-api-key: <api_key>
```

Response

```
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
```

```
{}
```

Assertiones verificadas

- `status` igual a `404`
- `body` igual a `{}` (objeto vacío)

Resultado: PASS

TC-API-012 · GET /users/abc — ID no numérico → 404

Archivo: `api/automated-tests/GET.cy.js`

Request

```
GET https://reqres.in/api/users/abc
x-api-key: <api_key>
```

Response

```
HTTP/1.1 404 Not Found
```

```
{}
```

Assertiones verificadas

- `status` igual a `404`

Resultado: PASS

TC-API-013 · PUT /users/2 — Actualización completa

Archivo: `api/automated-tests/PUT.cy.js`

Request

```
PUT https://reqres.in/api/users/2
Content-Type: application/json
x-api-key: <api_key>
```

```
{
  "name": "Updated User",
  "job": "Senior QA Engineer"
}
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
```

```
{
  "name": "Updated User",
  "job": "Senior QA Engineer",
  "updatedAt": "2026-06-22T14:31:14.780Z"
}
```

Assertiones verificadas

- `status` igual a `200`
- `body.name` igual a `"Updated User"`
- `body.job` igual a `"Senior QA Engineer"`
- `body.updatedAt` existe

Resultado: PASS

TC-API-014 · PATCH /users/2 — Actualización parcial

Archivo: `api/automated-tests/PATCH.cy.js`

Request

```
PATCH https://reqres.in/api/users/2
Content-Type: application/json
x-api-key: <api_key>
```

```
{  
  "job": "QA Lead"  
}
```

Response

```
HTTP/1.1 200 OK  
Content-Type: application/json; charset=utf-8
```

```
{  
  "job": "QA Lead",  
  "updatedAt": "2026-06-22T14:31:15.332Z"  
}
```

Assertiones verificadas

- `status` igual a `200`
- `body.job` igual a `"QA Lead"`
- `body.updatedAt` existe

Resultado: PASS

TC-API-015 · DELETE /users/2 — Eliminar usuario

Archivo: `api/automated-tests/DELETE.cy.js`

Request

```
DELETE https://reqres.in/api/users/2  
x-api-key: <api_key>
```

Response

```
HTTP/1.1 204 No Content
```

(sin cuerpo de respuesta)

Assertiones verificadas

- `status` igual a `204`

Resultado: PASS

Observaciones generales

Observación	Detalle
Autenticación requerida	ReqRes exige el header <code>x-api-key</code> desde 2024. Sin él retorna <code>401 Unauthorized</code> . La key se configura en <code>.env</code> y se inyecta automáticamente vía el comando custom <code>cy.reqresRequest()</code> .
Persistencia de datos	ReqRes no persiste datos creados. Los IDs del dataset mock van del 1 al 12. El POST genera un id en la respuesta pero el registro no queda en el sistema.
Validación de campos	El mock no valida campos requeridos ni tipos de datos. Los casos TC-API-005 a TC-API-008 documentan este comportamiento explícitamente.
Comportamiento real vs mock	En una API de producción, los casos con body vacío, campos faltantes o tipos incorrectos deberían retornar <code>400</code> o <code>422</code> . Las aserciones están adaptadas al comportamiento real del mock.

Reporte de ejecución automatizado

El reporte HTML generado por mochwesome contiene los resultados de ejecución, tiempos por caso y screenshots de fallos (si los hubiera).

Para generarlo:

```
npm test
npm run report
```

El reporte queda en: `cypress/reports/html/output.html`

Para convertirlo a PDF: abrir el archivo en el browser → `Ctrl + P` → **Guardar como PDF**.

Documento generado como parte de la prueba técnica QA Engineer — Makers